Лабораторная работа 2

Реализация структуры Data Vault

Надточиев А.Ф.
Группа 223-322
Дисциплина Хранилища данных
12 октября 2026 года

Цель и предметная область

Реализовать хранилище по принципам Data Vault: отделить бизнес-ключи, связи и описательные атрибуты; сохранять историю; обеспечить повторяемую загрузку и получить витрину текущего состояния.

Предметная область соответствует теме варианта 15: учёт проката на лыжной базе. Реализован контур клиентов, договоров проката и физических экземпляров инвентаря. Источник — условная система проката SKI_RENTAL. Все примеры записей вымышлены.

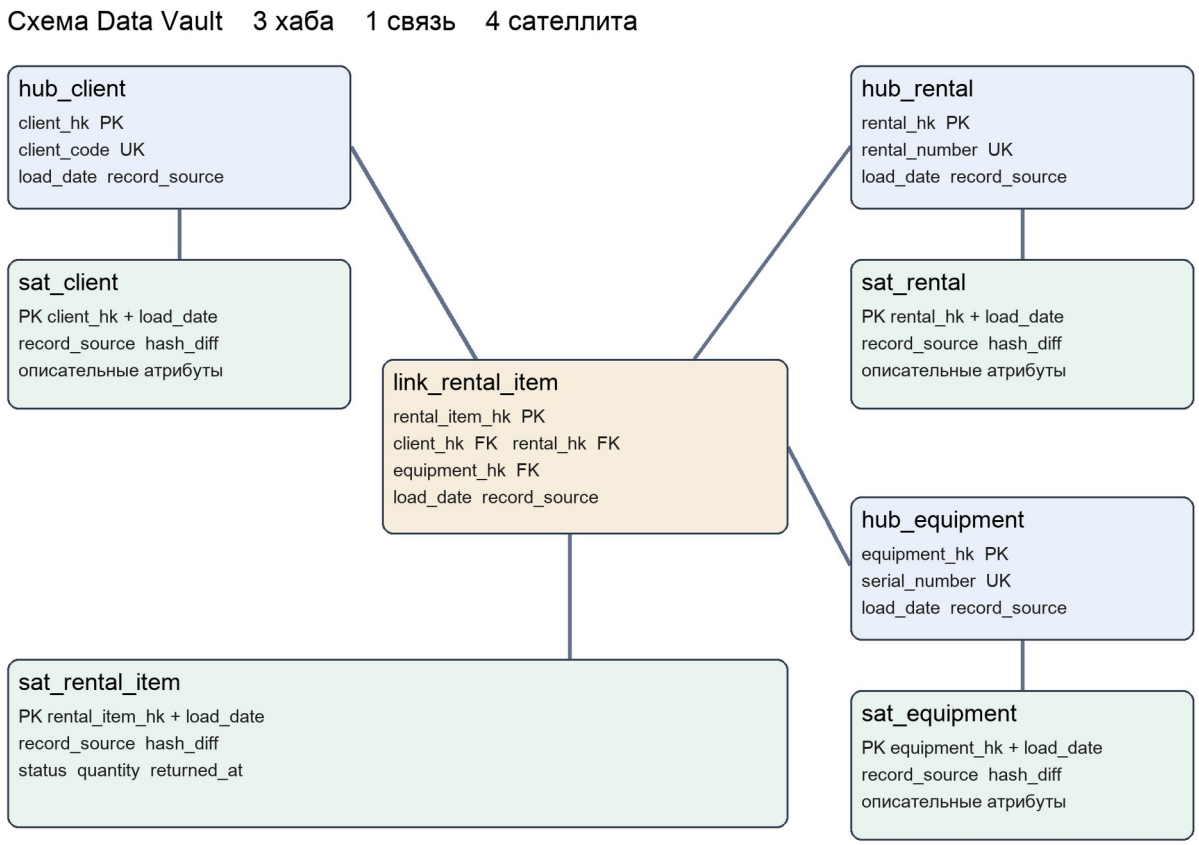
Бизнес ключи и границы модели

Сущность	Бизнес ключ	Значение
Клиент	client_code	Устойчивый код клиента CL-001
Договор проката	rental_number	Номер договора RENT-001
Экземпляр инвентаря	serial_number	Серийный номер NAD-SN-101
Позиция договора	Тройка ключей	Клиент + договор + экземпляр

Коды уникальны, неизменны и трактуются с учётом регистра. Имена клиентов, сегменты, названия инвентаря, статус и количество не входят в бизнес-ключи. Одна связь представляет позицию договора проката. Модель включает один источник и не моделирует продление договора отдельной сущностью.

Среда проверки: PostgreSQL 16.14. Для SHA256 используется стандартное расширение pgcrypto. На одну партию приходится один согласованный снимок; партии поступают по возрастанию времени загрузки.

Структура хранилища



Хабы содержат хеш бизнес-ключа, сам бизнес-ключ, время первой загрузки и источник. Связь хранит три внешних ключа хабов и собственный хеш. Каждый сателлит имеет одного родителя; первичный ключ сателлита состоит из ключа родителя и времени загрузки.

Сателлит	**Описательные атрибуты**
sat_client	client_name, client_segment
sat_rental	rental_description
sat_equipment	equipment_name, purchase_date
sat_rental_item	status, quantity, returned_at

У всех сателлитов есть record_source и hash_diff. Для другого источника следует создавать отдельные сателлиты, чтобы контексты независимых источников не заменяли друг друга.

Реализация и загрузка

Схема создаётся файлом 01_schema.sql. Данные и проверки находятся в 02_data_and_checks.sql. Оба файла опубликованы в репозитории; ссылка приведена в источниках.

```
CREATE OR REPLACE FUNCTION hash_key(VARIADIC parts text[])
RETURNS text LANGUAGE sql IMMUTABLE STRICT AS $$
 SELECT encode(public.digest(array_to_json(parts)::text,
                     'sha256'), 'hex')
$$;
```

Сериализация массива в JSON сохраняет границы значений: наборы («a|b», «c») и («a», «b|c») получают разные хеши. NULL и пустая строка также различаются. Хеш из 64 шестнадцатеричных символов хранится как text.

```
CREATE TABLE sat_rental_item (
 rental_item_hk text REFERENCES link_rental_item,
 load_date timestamptz NOT NULL,
 record_source text NOT NULL, hash_diff text NOT NULL,
 status text, quantity integer, returned_at timestamptz,
 PRIMARY KEY(rental_item_hk,load_date)
);
```

1. Исходные строки загружаются в stage_rental_item без ограничений целостности.
2. Запись с отсутствующим бизнес-ключом или метаданными сохраняется в rejected_rental_item.
3. Из корректных записей рассчитываются хеши.
4. Сначала заполняются хабы, затем связь, затем сателлиты.
5. Описательный хеш сравнивается с последней версией родителя: новая строка добавляется только при изменении.
6. Представление mart_current_rental_item выбирает последние версии сателлитов.

Порядок загрузки и ON CONFLICT DO NOTHING защищают от дубликатов при повторном запуске. Сырые описательные значения не агрегируются и не пересчитываются. Статус и время возврата сохраняются как поступившие значения источника.

Результаты выполнения

Объект	Число строк
hub_client	3
hub_rental	3
hub_equipment	2
link_rental_item	4
sat_client	3
sat_rental	3
sat_equipment	2
sat_rental_item	6
rejected_rental_item	1
mart_current_rental_item	4

История позиции CL-001 / RENT-001 / NAD-SN-101 содержит три версии. Переход к прежнему состоянию сохранён как отдельное событие загрузки.

Время загрузки UTC	Статус	Количество
01.10.2026 10:00	issued	1
02.10.2026 10:00	returned	1
03.10.2026 10:00	issued	1

Проверки и вывод

Успешно выполнены проверки количества строк, внешних ключей, текущей версии и отсутствия дубликатов при повторной загрузке B1, B2 и B3. Все SQL-утверждения завершились без ошибок; получено ALL DATA VAULT CHECKS PASSED. Проверочный файл выполнен повторно целиком. SHA256 бизнес-ключа CL-001 совпал с независимым расчётом Python.

Реализованы три хаба, одна связь и четыре сателлита. История A → B → A сохраняется, повторная загрузка идемпотентна, неполная запись не нарушает целостность Raw Vault, а витрина показывает актуальное состояние.

Источники

Методические материалы: «1 лаба варианты.docx» и «1 лаба лекция.pdf», папка «Хранилище данных» на Яндекс Диске.

PostgreSQL 16 — pgcrypto: https://www.postgresql.org/docs/16/pgcrypto.html

Код и фактический протокол: https://github.com/vasiliyvasilek20/data-warehouse-labs-223-322

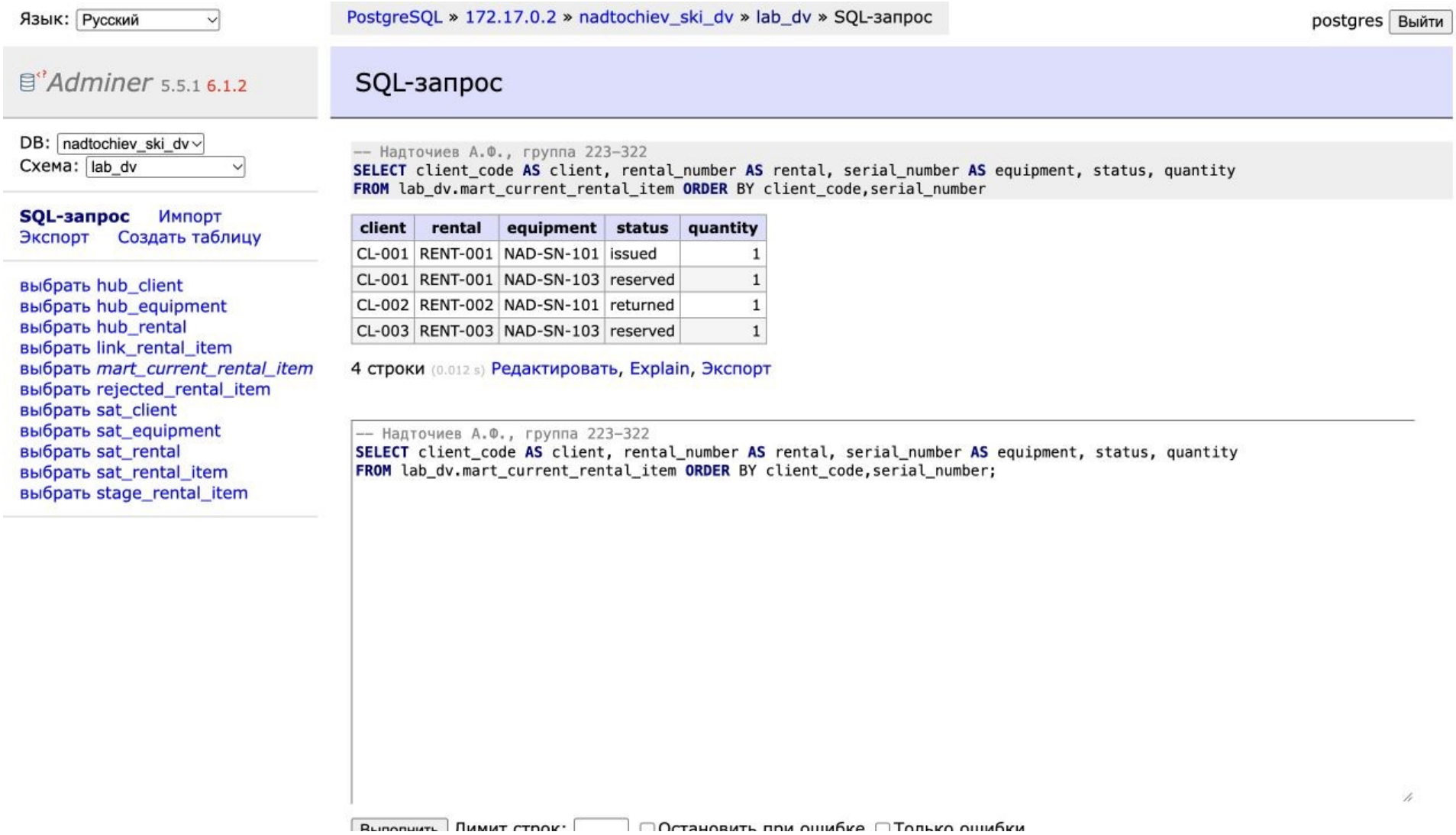
Скриншоты выполнения 1

Таблицы хранилища Data Vault



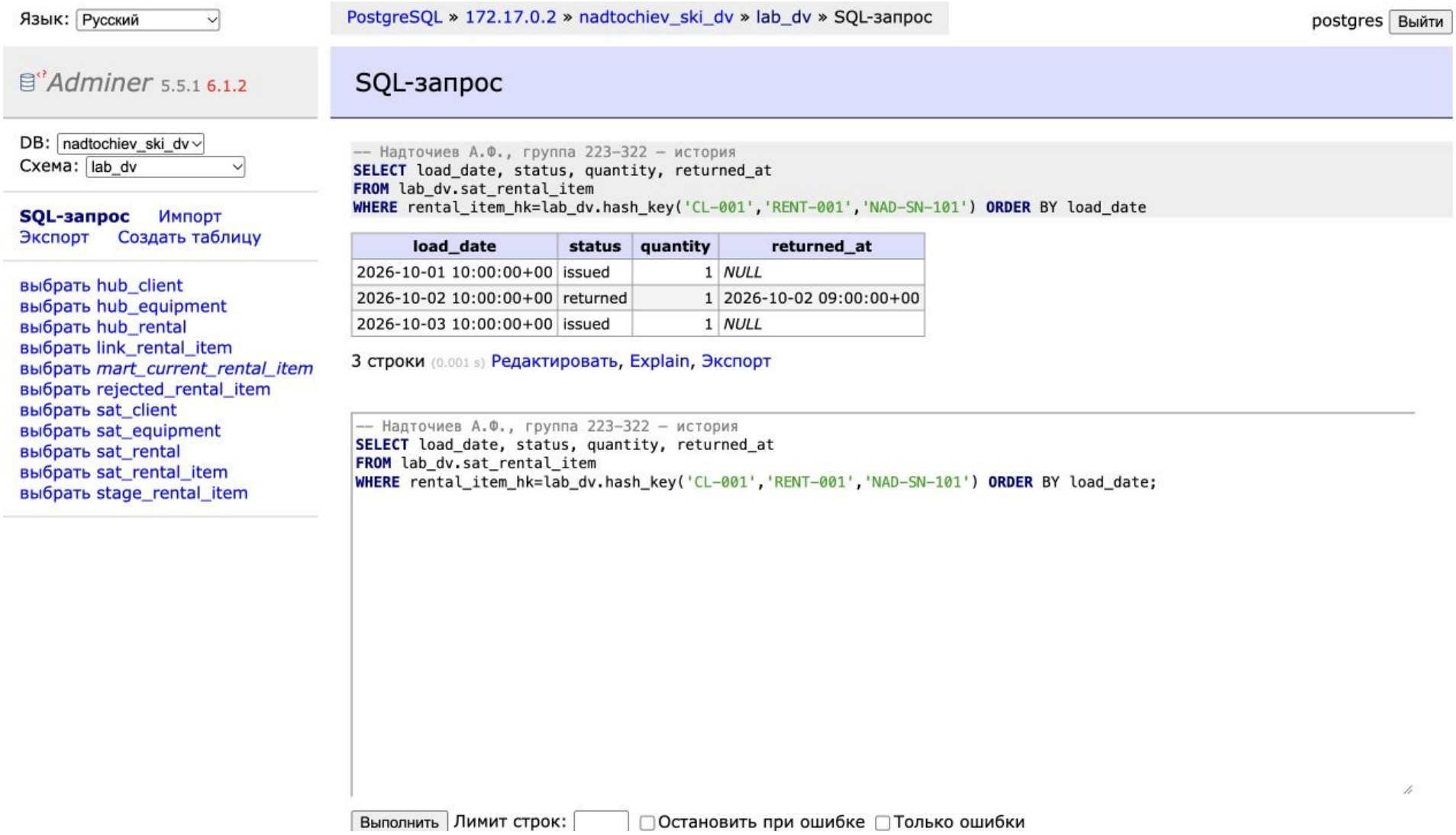
Скриншоты выполнения 2

Текущая витрина проката



Скриншоты выполнения 3

История позиции: issued → returned → issued



Скриншоты выполнения 4

Повторная загрузка и контрольные количества

